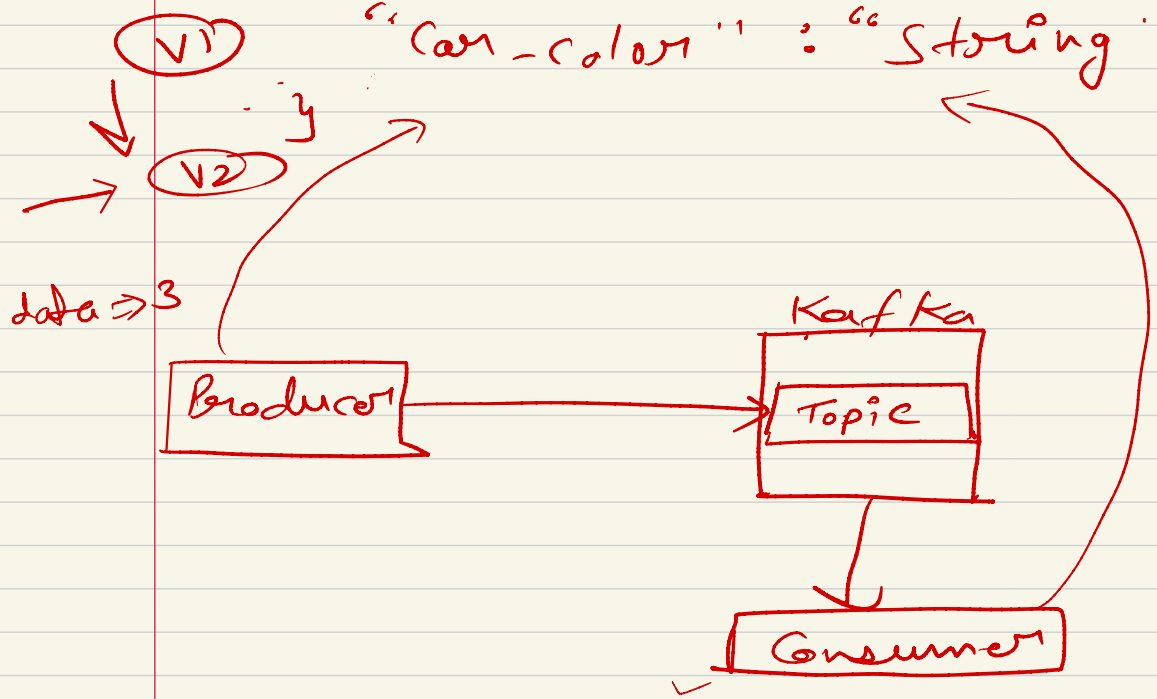
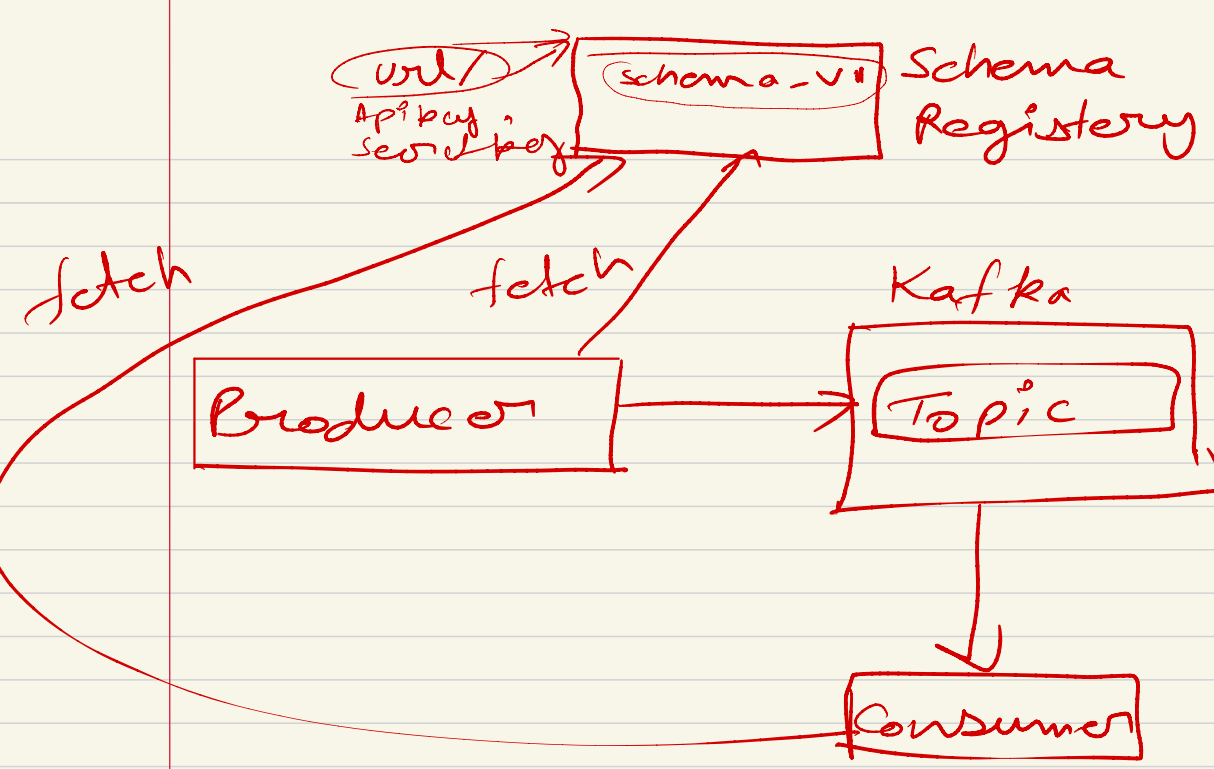
# Schema Registry in Confluent

data
```
{
  "car_name": "Creta",
  "car_color": "White"
}
```

schema
```
{
  "car_name": "String"
  "car_color": "String"
}
```




url
Api key
Secret key
schema_v1
Schema Registery
fetch
fetch
Kafka
Producer
Topic
Consumer

# Apache Kafka Interview Questions

## Q.1- Mention what Apache Kafka is?

Apache Kafka is a publish-subscribe messaging system developed by Apache written in Scala. It is a distributed, partitioned and replicated log service.

## Q.2- Mention what is the traditional method of message transfer?

The traditional method of message transfer includes two methods

- Queuing: In a queuing, a pool of consumers may read message from the server and each message goes to one of them
- Publish-Subscribe: In this model, messages are broadcasted to all consumers

Kafka caters single consumer abstraction that generalized both of the above- the consumer group.

## Q.3- Mention what are the benefits of Apache Kafka over the traditional technique?

Apache Kafka has following benefits above traditional messaging technique

- Fast: A single Kafka broker can serve thousands of clients by handling megabytes of reads and writes per second
- Scalable: Data are partitioned and streamlined over a cluster of machines to enable larger data
- Durable: Messages are persistent and is replicated within the cluster to prevent data loss
- Distributed by Design: It provides fault tolerance guarantees and durability

## Q.4- Mention what is the meaning of Broker in Kafka?

In the Kafka cluster, broker term is used to refer to Server.

## Q.5- Mention what is the Maximum Size of the Message does Kafka server can Receive?

The maximum size of the message that Kafka server can receive is 1000000 bytes.

## Q.6- Explain what is Zookeeper in Kafka and can we use Kafka without Zookeeper?

Zookeeper is an open source, high-performance co-ordination service used for distributed applications adapted by Kafka. No, it is not possible to bye-pass Zookeeper and connect straight to the Kafka broker. Once the Zookeeper is down, it cannot serve client requests. Zookeeper is basically used to communicate between different nodes in a cluster In Kafka, it is used to commit offset, so if node fails in any case it can be retrieved from the previously committed offset Apart from this it also does other activities like leader detection, distributed synchronization, configuration management, identifies when a new node leaves or joins, the cluster, node status in real time, etc.

## Q.7- Explain how messages are consumed by consumers in Kafka?

Transfer of messages in Kafka is done by using sendfile API. It enables the transfer of bytes from the socket to disk via kernel space saving copies and calls between kernel users back to the kernel.

## Q.8- Explain how you can improve the throughput of a remote consumer?

If the consumer is located in a different data centre from the broker, you may require to tune the socket buffer size to amortize the long network latency.

## Q.9- Explain how you can get Exactly Once Messaging from Kafka during data production?

During data production to get exactly once messaging from Kafka you have to follow two things: avoiding duplicates during data consumption and avoiding duplication during data production. Here are the two ways to get exactly one semantics while

data production: Avail a single writer per partition, every time you get a network error checks the last message in that partition to see if your last write succeeded In the message include a primary key (UUID or something) and de-duplicate on the consumer

## Q.10- Explain how you can reduce churn in Isr and when does Broker leave the Isr?

ISR is a set of message replicas that are completely synced up with the leaders, in other word ISR has all messages that are committed. ISR should always include all replicas until there is a real failure. A replica will be dropped out of ISR if it deviates from the leader.

## Q.11- Why Replication is required in Kafka?

Replication of messages in Kafka ensures that any published message does not lose and can be consumed in case of machine error, program error or more common software upgrades.

## Q.12- What does it indicate if a replica stays out of Isr for a long time?

If a replica remains out of ISR for an extended time, it indicates that the follower is unable to fetch data as fast as data accumulated at the leader.

## Q.13- Mention what happens if the preferred replica is not in the Isr?

If the preferred replica is not in the ISR, the controller will fail to move leadership to the preferred replica.

## Q.14- Is it possible to get the Message Offset after Producing?

You cannot do that from a class that behaves as a producer like in most queue systems, its role is to fire and forget the messages. The broker will do the rest of the work like appropriate metadata handling with id's, offsets, etc. As a consumer of the message, you can get the offset from a Kafka broker. If you look in the SimpleConsumer class, you will notice it fetches MultiFetchResponse objects that include offsets as a list. In addition to that, when you iterate the Kafka Message, you

will have MessageAndOffset objects that include both, the offset and the message sent.

## Q.15- Mention what is the difference between Apache Kafka and Apache Storm?

Apache Kafka: It is a distributed and robust messaging system that can handle huge amounts of data and allows passage of messages from one end-point to another. Apache Storm: It is a real time message processing system, and you can edit or manipulate data in real time. Apache storm pulls the data from Kafka and applies some required manipulation.

## Q.16- List the various components in Kafka?

The four major components of Kafka are:

- Topic – a stream of messages belonging to the same type
- Producer – that can publish messages to a topic
- Brokers – a set of servers where the publishes messages are stored
- Consumer – that subscribes to various topics and pulls data from the brokers.

## Q.17- Explain the role of the Offset?

Messages contained in the partitions are assigned a unique ID number that is called the offset. The role of the offset is to uniquely identify every message within the partition.

## Q.18- Explain the concept of Leader and Follower?

Every partition in Kafka has one server which plays the role of a Leader, and none or more servers that act as Followers. The Leader performs the task of all read and write requests for the partition, while the role of the Followers is to passively replicate the leader. In the event of the Leader failing, one of the Followers will take on the role of the Leader. This ensures load balancing of the server.

## Q.19- How do you define a Partitioning Key?

Within the Producer, the role of a Partitioning Key is to indicate the destination partition of the message. By default, a hashing-based Partitioner is used to determine the partition ID given the key. Alternatively, users can also use customized Partitions.

## Q.20- In the Producer when does Queuefullexception occur?

QueueFullException typically occurs when the Producer attempts to send messages at a pace that the Broker cannot handle. Since the Producer doesn't block, users will need to add enough brokers to collaboratively handle the increased load.

## Q.21- Explain the role of the Kafka Producer Api.

The role of Kafka's Producer API is to wrap the two producers – kafka.producer.SyncProducer and the kafka.producer.async.AsyncProducer. The goal is to expose all the producer functionality through a single API to the client.

# Apache Kafka Interview Questions

## (Q.1) What is Apache Kafka?

Ans. Apache Kafka is a publish-subscribe open source message broker application. This messaging application was coded in “Scala”. Basically, this project was started by the Apache software. Kafka’s design pattern is mainly based on the transactional logs design. For detailed understanding of Kafka, go through.

## (Q.2) Enlist the several components in Kafka.

Ans. The most important elements of Kafka are:

- Topic – Kafka Topic is a bunch or a collection of messages.
- Producer – In Kafka, Producers issue communications as well as publish messages to a Kafka topic.
- Consumer – Kafka Consumers subscribes to a topic(s) and also reads and processes messages from the topic(s).
- Brokers –While it comes to manage storage of messages in the topic(s) we use Kafka Brokers. For detailed understanding of Kafka components, go through,

## (Q.3) Explain the role of the offset.

Ans. There is a sequential ID number given to the messages in the partitions what we call an offset. So, to identify each message in the partition uniquely, we use these offsets.

## (Q.4) What is a Consumer Group?

Ans. The concept of Consumer Groups is exclusive to Apache Kafka. Basically, every Kafka consumer group consists of one or more consumers that jointly consume a set of subscribed topics.

## (Q.5) What is the role of the ZooKeeper in Kafka?

Ans. Apache Kafka is a distributed system built to use Zookeeper. Zookeeper's main role here is to build coordination between different nodes in a cluster. However, we also use Zookeeper to recover from previously committed offset if any node fails because it works as a periodic commit offset.

## (Q.6) Is it possible to use Kafka without ZooKeeper?

Ans. It is impossible to bypass Zookeeper and connect directly to the Kafka server, so the answer is no. If somehow, ZooKeeper is down, then it is impossible to service any client request.

## (Q.7) What do you know about Partition in Kafka?

Ans. In every Kafka broker, there are few partitions available. And, here each partition in Kafka can be either a leader or a replica of a topic.

## (Q.8) Why is Kafka technology significant to use?

Ans. There are some advantages of Kafka, which makes it significant to use:

- High-throughput : We do not need any large hardware in Kafka, because it is capable of handling high-velocity and high-volume data. Moreover, it can also support message throughput of thousands of messages per second.
- Low Latency : Kafka can easily handle these messages with the very low latency of the range of milliseconds, demanded by most of the new use cases.
- Fault-Tolerant : Kafka is resistant to node/machine failure within a cluster.
- Durability : As Kafka supports message replication, messages are never lost. It is one of the reasons behind durability.
- Scalability : Kafka can be scaled-out, without incurring any downtime on the fly by adding additional nodes.

## (Q.9) What are the main APIs of Kafka?

Ans. Apache Kafka has 4 main APIs:

- Producer API
- Consumer API
- Streams API
- Connector API

## (Q.10) What are consumers or users?

Ans. Mainly, Kafka Consumer subscribes to a topic(s), and also reads and processes messages from the topic(s). Moreover, with a consumer group name, Consumers label themselves. In other words, within each subscribing consumer group, each record published to a topic is delivered to one consumer instance. Make sure it is possible that Consumer instances can be in separate processes or on separate machines.

## Q.11 Explain the concept of Leader and Follower.

Ans. In every partition of Kafka, there is one server which acts as the Leader, and none or more servers play the role as Followers.

## Q.12 What ensures load balancing of the server in Kafka?

Ans. The main role of the Leader is to perform the task of all read and write requests for the partition, whereas Followers passively replicate the leader. Hence, at the time of Leader failing, one of the Followers took over the role of the Leader. Basically, this entire process ensures load balancing of the servers.

## Q.13 What roles do Replicas and the ISR play?

Ans. Basically, a list of nodes that replicate the log is Replicas. Especially, for a particular partition. However, they are irrespective of whether they play the role of the Leader. In addition, ISR refers to In-Sync Replicas. On defining ISR, it is a set of message replicas that are synced to the leaders.

## Q.14 Why are Replications critical in Kafka?

Ans. Because of Replication, we can be sure that published messages are not lost and can be consumed in the event of any machine error, program error or frequent software upgrades.

## Q.15 If a Replica stays out of the ISR for a long time, what does it signify?

Ans. Simply, it implies that the Follower cannot fetch data as fast as data accumulated by the Leader.

## Q.16 What is the process for starting a Kafka server?

Ans. It is the very important step to initialize the ZooKeeper server because Kafka uses ZooKeeper.So, the process for starting a Kafka server is: In order to start the ZooKeeper server: `> bin/zookeeper-server-start.sh config/zookeeper.properties` Next, to start the Kafka server: `> bin/kafka-server-start.sh config/server.properties`

## Q.17 In the Producer, when does QueueFullException occur?

Ans. whenever the Kafka Producer attempts to send messages at a pace that the Broker cannot handle at that time QueueFullException typically occurs. However, to collaboratively handle the increased load, users will need to add enough brokers(servers, nodes), since the Producer doesn't block.

## Q.18 Explain the role of the Kafka Producer API.

Ans. An API which permits an application to publish a stream of records to one or more Kafka topics is what we call Producer API.

## Q.19 What is the main difference between Kafka and Flume?

Ans. The main difference between Kafka and Flume are: Types of tool

- Apache Kafka– As Kafka is a general-purpose tool for both multiple producers and consumers.
- Apache Flume– Whereas, Flume is considered as a special-purpose tool for specific applications.

Replication feature

- Apache Kafka– Kafka can replicate the events.
- Apache Flume- whereas, Flume does not replicate the events.

## Q.20 Is Apache Kafka a distributed streaming platform? If yes, what can you do with it?

Ans. Undoubtedly, Kafka is a streaming platform. It can help: To push records easily. Also, can store a lot of records without giving any storage problems Moreover, it can process the records as they come in

## Q. 21 What can you do with Kafka?

Ans. It can perform in several ways, such as:

- In order to transmit data between two systems, we can build a real-time stream of data pipelines with it.
- Also, we can build a real-time streaming platform with Kafka, that can actually react to the data.

## Q.22 What is the purpose of the retention period in the Kafka cluster?

Ans. However, the retention period retains all the published records within the Kafka cluster. It doesn't check whether they have been consumed or not. Moreover, the records can be discarded by using a configuration setting for the retention period. And, it results as it can free up some space.

## Q.23 Explain the maximum size of a message that can be received by the Kafka?

Ans. The maximum size of a message that can be received by the Kafka is approx. 1000000 bytes.

## Q.24 What are the types of traditional method of message transfer?

Ans. Basically, there are two methods of the traditional message transfer method, such as:

- Queuing: It is a method in which a pool of consumers may read a message from the server and each message goes to one of them.
- Publish-Subscribe: Whereas in Publish-Subscribe, messages are broadcasted to all consumers.

## Q.25 What does ISR stand for in the Kafka environment?

Ans. ISR refers to In sync replicas. These are generally classified as a set of message replicas which are synced to be leaders.

## Q.26 What is Geo-Replication in Kafka?

Ans. For our cluster, Kafka MirrorMaker offers geo-replication. Basically, messages are replicated across multiple data centres or cloud regions, with MirrorMaker. So, it can be used in active/passive scenarios for backup and recovery; or also to place data closer to our users, or support data locality requirements.

## Q.27 Explain Multi-tenancy?

Ans. We can easily deploy Kafka as a multi-tenant solution. However, by configuring which topics can produce or consume data, Multi-tenancy is enabled. Also, it provides operations support for quotas.

## Q.28 What is the role of Consumer API?

Ans. An API which permits an application to subscribe to one or more topics and also to process the stream of records produced to them is what we call Consumer API.

## Q.29 Explain the role of Streams API?

Ans. An API which permits an application to act as a stream processor, and also consuming an input stream from one or more topics and producing an output stream to one or more output topics, moreover, transforming the input streams to output streams effectively, is what we call Streams API.

## Q.30 What is the role of Connector API?

Ans. An API which permits to run as well as build the reusable producers or consumers which connect Kafka topics to existing applications or data systems is what we call the Connector API.

## Q.31 Explain Producer?

Ans. The main role of Producers is to publish data to the topics of their choice. Basically, its duty is to select the record to assign to partition within the topic.

## Q.32 Compare: RabbitMQ vs Apache Kafka

Ans. One of Apache Kafka's alternatives is RabbitMQ. So, let's compare both:

- Features
    - Apache Kafka– Kafka is distributed, durable and highly available, here the data is shared as well as replicated.
    - RabbitMQ– There are no such features in RabbitMQ.
- Performance rate
    - Apache Kafka– To the tune of 100,000 messages/second.
    - RabbitMQ- In case of RabbitMQ, the performance rate is around 20,000 messages/second.

## Q.33 Compare: Traditional queuing systems vs Apache Kafka

Ans. Let's compare Traditional queuing systems vs Apache Kafka feature-wise:
Messages Retaining

- Traditional queuing systems– It deletes the messages just after processing completion typically from the end of the queue.
- Apache Kafka– But in Kafka, messages persist even after being processed. That implies messages in Kafka don't get removed as consumers receive them. Logic-based processing
- Traditional queuing systems–Traditional queuing systems don't permit processing logic based on similar messages or events.
- Apache Kafka– Kafka permits to process logic based on similar messages or events.

## Q.34 Why Should we use Apache Kafka Cluster?

Ans. In order to overcome the challenges of collecting the large volume of data, and analyzing the collected data we need a messaging system. Hence Apache Kafka came into the story. Its benefits are:

- It is possible to track web activities just by storing/sending the events for real-time processes.
- Through this, we can Alert as well as report the operational metrics.
- Also, we can transform data into the standard format. *Moreover, it allows continuous processing of streaming data to the topics.
- Due to its wide use, it is ruling over some of the most popular applications like ActiveMQ, RabbitMQ, AWS etc.

## Q.35 Explain the term "Log Anatomy".

Ans. We view logs as the partitions. Basically, a data source writes messages to the log. One of the advantages is, at any time one or more consumers read from the log they select.

## Q.36 What is a Data Log in Kafka?

Ans. As we know, messages are retained for a considerable amount of time in Kafka. Moreover, there is flexibility for consumers that they can read as per their convenience. Although, there is a possible case that if Kafka is configured to keep messages for 24 hours and possibly that time the consumer is down for a time greater than 24 hours, then the consumer may lose those messages. However, still, we can read those messages from the last known offset, but only at a condition that the downtime on part of the consumer is just 60 minutes. Moreover, on what consumers are reading from a topic Kafka doesn't keep state.

## Q.37 Explain how to Tune Kafka for Optimal Performance.

Ans. So, ways to tune Apache Kafka it is to tune its several components:

- Tuning Kafka Producers
- Kafka Brokers Tuning
- Tuning Kafka Consumers

## Q.38 State Disadvantages of Apache Kafka.

Ans. Limitations of Kafka are:

- No Complete Set of Monitoring Tools
- Issues with Message Tweaking
- Not support wildcard topic selection
- Lack of Pace

## Q.39 Enlist all Apache Kafka Operations.

Ans. Apache Kafka Operations are:

- Addition and Deletion of Kafka Topics
- How to modify the Kafka Topics
- Distinguished Turnoff
- Mirroring Data between Kafka Clusters
- Finding the position of the Consumer
- Expanding Your Kafka Cluster
- Migration of Data Automatically
- Retiring Servers
- Data Centers

## Q.40 Explain Apache Kafka Use Cases?

Ans. Apache Kafka has so many use cases, such as:

- Kafka Metrics It is possible to use Kafka for operational monitoring data. Also, to produce centralized feeds of operational data, it involves aggregating statistics from distributed applications.
- Kafka Log Aggregation Moreover, to gather logs from multiple services across an organization.
- Stream Processing While stream processing, Kafka's strong durability is very useful.

## Q.41 Some of the most notable applications of Kafka.

Ans. Some of the real-time applications are:

- Netflix
- Mozilla
- Oracle

## Q.42 Features of Kafka Stream.

Ans. Some best features of Kafka Stream are

- Kafka Streams are highly scalable and fault-tolerant.
- Kafka deploys to containers, VMs, bare metal, cloud.
- We can say, Kafka streams are equally viable for small, medium, & large use cases.
- Also, it is fully in integration with Kafka security.
- Write standard Java applications.
- Exactly-once processing semantics.
- Moreover, there is no need for a separate processing cluster.

## Q.43 What do you mean by Stream Processing in Kafka?

Ans. The type of processing of data continuously, real-time, concurrently, and in a record-by-record fashion is what we call Kafka Stream processing.

## Q.44 What are the types of System tools?

Ans. There are three types of System tools:

- Kafka Migration Tool It helps to migrate a broker from one version to another.
- Mirror Maker Mirror Maker tool helps to offer a mirror of one Kafka cluster to another.
- Consumer Offset Checker For the specified set of Topics as well as Consumer Group, it shows Topic, Partitions, Owner.

## Q.45 What are Replication Tools and their types?

Ans. For the purpose of stronger durability and higher availability, a replication tool is available here. Its types are −

- Create Topic Tool
- List Topic Tool
- Add Partition Tool

## Q.46 What is the Importance of Java in Apache Kafka?

Ans. For the need of the high processing rates that come standard on Kafka, we can use the Java language. Moreover, for Kafka consumer clients also, Java offers good community support. So, we can say it is the right choice to implement Kafka in Java.

## Q.47 State one best feature of Kafka.

Ans. The best feature of Kafka is “Variety of Use Cases”. It means Kafka is able to manage the variety of use cases which are very common for a Data Lake. For Example log aggregation, web activity tracking, and so on.

## Q.48 Explain the term “Topic Replication Factor”.

Ans. It is very important to factor in topic replication while designing a Kafka system. Hence, if in any case, a broker goes down its topics’ replicas from another broker can solve the crisis.

## Q.49 Explain some Kafka Streams real-time Use Cases.

Ans. So, the use cases are:

- The New York Times: This company uses it to store and distribute, in real-time, published content to the various applications and systems that make it available to the readers. Basically, it uses Apache Kafka and the Kafka Streams both.
- Zalando: As an ESB (Enterprise Service Bus) as the leading online fashion retailer in Europe Zalando uses Kafka.
- LINE: Basically, to communicate to one another LINE application uses Apache Kafka as a central data hub for their services.

## Q.50 What are Guarantees provided by Kafka?

Ans. They are:

- The order will be the same for both the Messages sent by a producer to a particular topic partition. That
- Moreover, the consumer instance sees records in the order in which they are stored in the log.
- Also, we can tolerate up to N-1 server failures, even without losing any records committed to the log.

# Kafka Interview Questions

## Easy Category:

Q.1: What is Apache Kafka?
Ans: Apache Kafka is a distributed streaming platform that allows for the publishing and subscribing of streams of records.

Q.2: What are the key components of Kafka?
Ans: Kafka consists of topics, producers, consumers, brokers, and ZooKeeper (used for coordination and synchronisation).

Q.3: What is a Kafka topic?
Ans: A Kafka topic is a category or feed name to which records are published. It is divided into partitions for scalability and parallelism.

Q.4: Explain the role of Kafka producers.
Ans: Kafka producers are responsible for publishing messages to Kafka topics. They write records to Kafka brokers.

Q.5: What are Kafka consumers?
Ans: Kafka consumers subscribe to topics and consume records published to them. They read records from Kafka brokers.

Q.6:What is a Kafka broker?
Ans: A Kafka broker is a server that manages the storage and replication of Kafka topics. It is responsible for handling read and write requests.

Q.7:How does Kafka ensure fault tolerance?
Ans: Kafka achieves fault tolerance through replication. Each partition of a topic is replicated across multiple brokers.

Q.8:What is the role of ZooKeeper in Kafka?
Ans: ZooKeeper is used for managing the coordination and synchronization of Kafka brokers. It maintains the metadata and tracks the status of brokers.

Q.9:Explain the concept of message offset in Kafka.
Ans: Message offset represents the unique identifier of a message within a partition. It helps in tracking the position of a consumer in a partition.

Q.10:How does Kafka handle data retention?
Ans: Kafka has a configurable retention period for topics. It can retain messages for a certain duration or based on a certain message size.

Q.11:How can you monitor Kafka cluster health?
Ans: Kafka provides various metrics that can be monitored using tools like Kafka Manager, Confluent Control Center, or custom monitoring solutions.

Q.12:What is a Kafka consumer offset?
Ans: A consumer offset represents the position of a consumer within a partition. It tracks the last consumed message offset.

Q.13:Explain the role of the Kafka log compaction process.
Ans: The log compaction process in Kafka removes older, duplicate messages from a partition while retaining the latest message for each unique key.

Q.14:How can you control the number of consumers in a consumer group?
Ans: The number of consumers in a consumer group can be controlled by adding or removing consumer instances from the group.

Q.15:What is the purpose of Kafka Connect transformations?
Ans: Kafka Connect transformations allow for data manipulation during the process of moving data between Kafka and external systems.

Q.16:How does Kafka handle data compression?
Ans: Kafka supports various compression codecs, such as GZIP, Snappy, and LZ4, which can be configured to compress the messages before storage.

Q.17:What is a Kafka topic partition?
Ans: A Kafka topic partition is a logical division of a topic that allows for parallelism and scalability in handling message processing.

Q.18:Explain the concept of Kafka message durability.
Ans: Kafka ensures message durability by writing messages to disk before acknowledging the producers. This prevents data loss in case of failures.

Q.19:How can you ensure message delivery order across multiple Kafka partitions?
Ans: Kafka doesn't guarantee message delivery order across partitions. However, you can use a single partition or key-based partitioning to maintain order within a partition.

Q.20:What is the purpose of Kafka consumer offset commit intervals?
Ans: The offset commit interval determines how frequently the consumer commits its current offset. A shorter interval provides lower latency but higher commit overhead.

Q.21:What is the role of Kafka brokers in a Kafka cluster?
Ans: Kafka brokers are responsible for storing and serving the published messages. They handle read and write requests from producers and consumers.

Q.22:How does Kafka ensure high throughput and low latency?
Ans: Kafka achieves high throughput and low latency by utilizing a distributed architecture, parallel processing, and efficient disk I/O operations.

Q.23:Explain the concept of Kafka consumer offsets commit strategies.
Ans: Kafka consumer offsets can be committed using manual commit or automatic commit strategies. Manual commit gives more control, while automatic commit is simpler.

Q.24:How can you scale Kafka consumers?
Ans: Kafka consumers can be scaled horizontally by adding more consumer instances to a consumer group, allowing for parallel processing and increased throughput.

Q.25:What is the purpose of Kafka message serialisation?
Ans: Kafka requires messages to be serialised before being sent. Serialisation converts messages from their object representation to a format that can be transmitted.

Q.26:How does Kafka handle data retention policy enforcement?
Ans: Kafka enforces the data retention policy by periodically checking the timestamp or size of messages in a topic and removing them if they exceed the configured limits.

Q.27:Explain the role of Kafka ZooKeeper in consumer coordination.
Ans: Kafka ZooKeeper is used for consumer coordination, such as tracking consumer group membership, partition ownership, and leader election.

Q.28:How can you handle duplicate messages in Kafka consumers?
Ans: Kafka consumers can handle duplicate messages by deduplicating based on unique identifiers or using idempotent processing logic to handle duplicate records.

Q.29:What is the purpose of Kafka record headers?
Ans: Kafka record headers allow for attaching additional metadata to Kafka messages, enabling flexible processing and enrichment of messages.

Q.30:How can you monitor Kafka consumer lag?
Ans: Kafka consumer lag can be monitored by comparing the current consumer offset with the latest available offset for each partition using Kafka consumer lag monitoring tools.

## Medium Category:

Q.1:What is the role of a Kafka producer callback?
Ans: A producer callback is invoked after a record is sent to Kafka. It allows for handling success or failure notifications.

Q.2:How does Kafka handle backpressure?
Ans: Kafka employs backpressure at the consumer end by allowing consumers to control their own consumption rates.

Q.3:What is the significance of the Kafka Connect API?
Ans: Kafka Connect API is used for building and running connectors that move data in and out of Kafka. It provides a scalable and fault-tolerant solution for data integration.

Q.4:Explain the concept of a Kafka consumer group.
Ans: A consumer group is a group of consumers that work together to consume records from Kafka topics. Each consumer in a group processes a subset of partitions.

Q.5:How does Kafka handle data replication?
Ans: Kafka uses a leader-follower replication model. Each partition has one leader and multiple followers, ensuring data redundancy and fault tolerance.

Q.6:What is the purpose of the Kafka Streams library?
Ans: Kafka Streams is a client library for building stream processing applications that can process and analyze real-time data from Kafka topics.

Q.7:How can you ensure message ordering within a Kafka partition?
Ans: Kafka guarantees message ordering within a partition. Multiple messages from the same producer will be written to the same partition and processed in order.

Q.8:What is a Kafka offset commit?
Ans: Offset commit is the process of acknowledging the successful consumption of messages by a Kafka consumer. It helps in tracking the progress of consumers.

Q.9:Explain the concept of message retention policies in Kafka.
Ans: Kafka provides two retention policies: log compaction and delete. Log compaction retains the latest message for each key, while delete retains messages based on time or size.

Q.10:How does Kafka handle failover scenarios?
Ans: In case of broker failure, Kafka ensures failover by electing a new leader for affected partitions. Consumers automatically recover and continue from the last committed offset.

Q.11:How can you handle Kafka message retries in case of failures?
Ans: You can configure the Kafka producer to retry failed messages using the "retries" configuration parameter. Additionally, you can set a callback to handle failures.

Q.12:Explain the concept of Kafka log compaction cleanup policy.
Ans: The log compaction cleanup policy in Kafka determines when to remove obsolete messages during log compaction. It can be based on time or log size.

Q.13:How does Kafka handle consumer rebalancing?
Ans: Kafka rebalances consumers in a consumer group by redistributing partitions among the consumers when new consumers join or existing consumers leave the group.

Q.14:What is the role of Kafka Streams windowing operations?
Ans: Kafka Streams windowing operations allow for aggregating data over fixed time intervals or key-specific session windows, enabling time-based analysis.

Q.15:How can you handle schema evolution in Kafka using Apache Avro?
Ans: Apache Avro, a schema-based serialisation framework, enables schema evolution by supporting backward and forward compatibility when reading and writing data.

Q.16:Explain the purpose of Kafka Connect converters.
Ans: Kafka Connect converters are responsible for serialising and deserializing data between Kafka and external systems. They handle data format conversions.

Q.17:How does Kafka ensure data consistency and durability?
Ans: Kafka ensures data consistency and durability through replication, which replicates each partition across multiple brokers, ensuring fault tolerance.

Q.18:What is the role of Kafka Streams Interactive Queries Server?
Ans: The Kafka Streams Interactive Queries Server provides a REST API for querying the state stores maintained by Kafka Streams applications.

Q.19:Explain the purpose of Kafka Streams stateful operations.
Ans: Kafka Streams stateful operations allow for maintaining and updating state while processing data streams, enabling complex processing tasks.

Q.20:How can you handle late-arriving events in Kafka Streams?
Ans: Kafka Streams provides windowing operations that allow you to define a grace period for late-arriving events, ensuring accurate processing.

Q.21:What is the purpose of Kafka Streams state store changelog topics?
Ans: Kafka Streams state stores use changelog topics to persist the local state of a stream processing application, ensuring fault tolerance and state recovery.

Q.22:How does Kafka handle leader elections in a broker cluster?
Ans: Kafka uses ZooKeeper for leader election in a broker cluster. When a leader fails, ZooKeeper helps elect a new leader for the affected partition.

Q.23:Explain the concept of Kafka message timestamping.

Ans: Kafka allows messages to be time stamped with a user-defined timestamp or the producer's system timestamp. It can be used for message ordering and time-based processing.

Q.24:How can you ensure message delivery guarantees in Kafka producers?
Ans: Kafka producers can ensure message delivery guarantees by configuring the "acks" parameter appropriately and handling producer errors and retries.

Q.25:What is the role of Kafka Connect converters in serialization and deserialization?
Ans: Kafka Connect converters handle the serialization and deserialization of data between Kafka and external systems, ensuring compatibility and data integrity.

Q.26:How does Kafka handle data partitioning across brokers?
Ans: Kafka uses consistent hashing to assign partitions to brokers, ensuring even distribution and fault tolerance by replicating partitions across multiple brokers.

Q.27:Explain the purpose of Kafka Streams processor API.
Ans: Kafka Streams processor API provides a low-level, fine-grained interface for building custom stream processors, enabling advanced stream processing operations.

Q.28:How can you handle dead-lettering in Kafka consumers?
Ans: Dead-lettering in Kafka consumers involves redirecting failed or unprocessable messages to a separate topic or storage for further analysis or manual intervention.

Q.29:What is the role of Kafka Connect in sink and source connectors?
Ans: Kafka Connect acts as a framework for building sink and source connectors that facilitate the movement of data between Kafka and external systems.

Q.30:How does Kafka handle message offsets for Kafka Streams?
Ans: Kafka Streams tracks message offsets internally, ensuring the processing of each message exactly once by maintaining the offset state for fault tolerance.

## Hard Category:

Q.1:How does Kafka handle data replication across multiple data centers?
Ans: Kafka allows for data replication across multiple data centers through the use of MirrorMaker or the built-in MirrorMaker 2.0.

Q.2:Explain the role of the Kafka Controller.
Ans: The Kafka Controller is responsible for managing and coordinating the actions of brokers in the Kafka cluster. It handles tasks such as leader election and partition reassignment.

Q.3:What is the purpose of the Kafka Streams DSL?
Ans: Kafka Streams DSL is a high-level abstraction for building stream processing applications. It provides an easy-to-use interface for defining data transformations and aggregations.

Q.4:How does Kafka handle semantics exactly once?
Ans: Kafka provides exactly once semantics through the use of idempotent producers and transactional producers. Consumers can also use the commitSync() method for exactly once processing.

Q.5:What is the purpose of Kafka Connect converters?
Ans: Kafka Connect converters are used to serialize and deserialize data between Kafka and external systems. They ensure compatibility and seamless integration.

Q.6:Explain the concept of Kafka log compaction.
Ans: Log compaction in Kafka retains only the latest message for each unique key in a topic. It is useful for scenarios where the latest state of a record is important.

Q.7:How does Kafka handle data rebalancing in consumer groups?
Ans: When a consumer joins or leaves a consumer group, Kafka triggers a rebalance process. It redistributes the partitions among the remaining consumers in a group.

Q.8:What is the role of Kafka ACLs (Access Control Lists)?
Ans: Kafka ACLs are used to control access to topics and perform authorization checks. They provide granular control over read, write, and admin operations.

Q.9:Explain the purpose of Kafka Streams Interactive Queries.
Ans: Kafka Streams Interactive Queries allow applications to query state stores created by stream processing tasks. It provides real-time access to the current state of the data.

Q.10:What is the role of the Kafka Connect framework?
Ans: Kafka Connect is a framework for building and running scalable and fault-tolerant data pipelines between Kafka and other systems. It provides a plugin-based architecture for easy extensibility.

Q.11:How does Kafka handle data compaction in the presence of tombstone messages?
Ans: Kafka uses tombstone messages (special delete markers) during log compaction to mark records for deletion. Tombstones take precedence over non-tombstone messages.

Q.12:Explain the concept of Kafka message format evolution using Schema Registry.
Ans: The Schema Registry in Kafka allows for message format evolution by supporting schema compatibility checks and providing a centralised schema repository.

Q.13:How can you handle out-of-order data in Kafka Streams processing?
Ans: Kafka Streams provides a time windowing feature that allows for handling out-of-order data by defining a window size that accommodates late-arriving events.

Q.14:What is the purpose of Kafka Streams state stores?
Ans: Kafka Streams state stores enable the storage and querying of application-specific state during stream processing, facilitating stateful transformations.

Q.15:How can you configure Kafka to ensure at least once message delivery semantics?
Ans: Kafka provides at least once message delivery semantics by enabling the "acks" configuration parameter and implementing proper error handling and retries.

Q.16:Explain the role of Kafka Connect converters in Avro serialisation.
Ans: Kafka Connect converters handle Avro serialisation/deserialization by interacting with the Avro schema registry to ensure compatibility and schema evolution.

Q.17:How does Kafka handle partition rebalancing during a consumer group rebalance?
Ans: Kafka follows a group coordination protocol, where the group coordinator orchestrates the partition assignment and rebalancing process during consumer group rebalances.

Q.18:What is the purpose of Kafka Streams state store suppliers?
Ans: State store suppliers in Kafka Streams define how the state stores are created and restored, allowing for customization and integration with external systems.

Q.19:Explain the concept of Kafka transactional messaging.
Ans: Kafka transactional messaging ensures atomicity and consistency by allowing producers and consumers to participate in transactions across multiple partitions.

Q.20:How can you handle stateful aggregations in Kafka Streams?
Ans: Kafka Streams provides built-in stateful aggregation operations, such as count, sum, and average, which can be used to maintain and update aggregations over time.

Q.21:What is the purpose of Kafka's log cleaner and log compaction?
Ans: Kafka's log cleaner is responsible for removing obsolete log segments, while log compaction retains the latest message for each unique key within a topic.

Q.22:Explain the concept of Kafka message compression.
Ans: Kafka supports message compression to reduce network bandwidth and storage costs. It offers configurable compression codecs like GZIP, Snappy, and LZ4.

Q.23:How can you handle transactional messaging in Kafka consumers?
Ans: Kafka consumers can participate in transactions by using the Kafka transactional API, which allows for atomic and consistent processing across multiple partitions.

Q.24:What is the purpose of Kafka Streams interactive queries?
Ans: Kafka Streams interactive queries enable applications to query the current state of the data stored in the state stores maintained by Kafka Streams.

Q.25:How does Kafka handle partition leadership rebalancing?
Ans: Kafka handles partition leadership rebalancing by electing a new leader when the current leader fails or when new brokers join or leave the cluster.

Q.26:Explain the concept of Kafka consumer group coordination protocol.
Ans: Kafka consumer group coordination protocol uses ZooKeeper or the newer group coordination protocol based on the Kafka brokers to manage consumer group membership and rebalancing.

Q.27:What is the role of Kafka Streams KTables?
Ans: Kafka Streams KTables represent the state of a stream at a specific point in time. They provide a materialized view of the data for efficient querying.

Q.28:How can you achieve near real-time processing with Kafka Streams?
Ans: Near real-time processing can be achieved with Kafka Streams by configuring appropriate time windows, reducing processing time, and optimizing the stream topology.

Q.29:Explain the purpose of Kafka Streams punctuators.
Ans: Kafka Streams punctuators allow for the insertion of user-defined processing logic at regular intervals, enabling periodic actions and time-driven transformations.

Q.30:What is the purpose of Kafka's high-level consumer API?
Ans: Kafka's high-level consumer API provides a simpler interface for building Kafka consumers by handling complex details such as partition assignment and offset management.

Q.31:What is the purpose of Kafka Connect converters in Avro serialization?
Ans: Kafka Connect converters are responsible for serializing and deserializing data between Kafka and external systems. In the case of Avro, converters interact with the Avro schema registry to ensure compatibility and schema evolution during serialization.

Q.32:How does Kafka handle message deduplication?
Ans: Kafka provides idempotent producers, which assign a unique identifier (message key) to each message. By using the message key, Kafka can identify and filter out duplicate messages during processing.

Q.33:Explain the concept of Kafka's Exactly Once Semantics.
Ans: Kafka's Exactly Once Semantics guarantees that messages are processed exactly once, without duplication or loss. This is achieved through the combination of idempotent producers, transactional producers, and transactional consumer offsets.

Q.34:What is the purpose of Kafka Connect Sinks and how are they used?
Ans: Kafka Connect Sinks are connectors that move data from Kafka to external systems. They enable the integration of Kafka with databases, file systems, and other data stores, allowing data to be consumed by other applications.

Q.35:How does Kafka Streams handle state restoration after a failure?
Ans: Kafka Streams leverages Kafka's built-in log compaction and changelog topics to persist and restore the state of stream processing applications. The application's state is continuously maintained and can be restored in case of failure.

Q.36:Explain the concept of Kafka Streams Global State Stores.
Ans: Kafka Streams Global State Stores are replicated and fault-tolerant storage mechanisms that allow all instances of a Kafka Streams application to access the same state. They enable global aggregations and lookups across all partitions.

Q.37:What is the purpose of Kafka's MirrorMaker tool?
Ans: Kafka's MirrorMaker tool is used for data replication between Kafka clusters. It allows data to be mirrored from one cluster to another, enabling data backup, disaster recovery, and data center migration scenarios.

Q.38:How does Kafka handle schema evolution in Avro serialization?
Ans: Kafka's Avro serialization, along with the Schema Registry, supports schema evolution by allowing compatible schema changes without breaking compatibility with existing data. This ensures backward and forward compatibility during data serialization.

Q.39:Explain the role of Kafka Streams in event-driven microservices architectures.
Ans: Kafka Streams provides a lightweight and scalable stream processing library that enables event-driven microservices architectures. It allows the processing and analysis of real-time data streams, facilitating event-driven communication and data processing.

Q.40:What are the considerations for scaling Kafka clusters and applications in a production environment?
Ans: Scaling Kafka clusters and applications involves considerations such as adding more brokers, increasing the number of partitions, optimizing hardware resources, and fine-tuning configuration parameters. Load balancing and monitoring tools are also essential for managing scalability effectively.

# Kafka Assignment

## Real-Time Data Processing with Confluent Kafka, MySQL, and Avro

**Objective:**
In this assignment, you will build a Kafka producer and a consumer group that work with a MySQL database, Avro serialization, and multi-partition Kafka topics. The producer will fetch incremental data from a MySQL table and write Avro serialized data into a Kafka topic. The consumers will deserialize this data and append it to separate JSON files.

**Tools Required:**

- Python 3.7 or later
- Confluent Kafka Python client
- MySQL Database
- Apache Avro
- A suitable IDE for coding (e.g., PyCharm, Visual Studio Code)

**Background:**
You are working in a fictitious e-commerce company called **"BuyOnline"** which has a MySQL database that stores product information such as product ID, name, category, price, and updated timestamp. The database gets updated frequently with new products and changes in product information. The company wants to build a real-time system to stream these updates incrementally to a downstream system for real-time analytics and business intelligence.

**Steps:**

**Step 1: MySQL Table Setup**
Create a table named 'product' in MySQL database with the following columns:

➔ id - INT (Primary Key)
➔ name - VARCHAR
➔ category - VARCHAR

➔ price - FLOAT
➔ last_updated - TIMESTAMP

**Step 2: Kafka Producer**

- Write a Kafka producer in Python that uses a MySQL connector to fetch data from the MySQL table.
- In your producer code, maintain a record of the last read timestamp. Each time you fetch data, use a SQL query to get records where the last_updated timestamp is greater than the last read timestamp.
- Serialize the data into Avro format and publish the data to a Kafka topic named "product_updates". This topic should be configured with 10 partitions.
- Use the product ID as the key when producing messages. This will ensure that all updates for the same product end up in the same partition.
- Update the last read timestamp after each successful fetch.

**Step 3: Kafka Consumer Group and Data Transformation**

- Write a Kafka consumer in Python and set it up as a consumer group of 5 consumers.
- Each consumer should read data from the "product_updates" topic.
- Deserialize the Avro data back into a Python object.
- Implement data transformation logic. For example:
- Change the category column to uppercase.
- Apply some business logic to update the price column, such as applying a discount if a particular product falls under a specific category.

**Step 4: Writing Transformed Data to JSON Files**

- Each consumer should convert the transformed Python object into a JSON string and append the JSON string to a separate JSON file. Make sure to open the file in append mode.
- Also, ensure each new record is written in a new line for ease of reading.
- Close the file after writing to make sure all data is saved properly.

**Deliverables:**

- The source code for the Kafka producer and consumer group in Python.
- SQL queries used for incremental data fetch from MySQL.
- Avro schema used for data serialization.
- Data transformation logic.
- The resulting JSON files with appended records.
- Documentation explaining how to run and test the system.
- Screenshots demonstrating the successful running of your application.

**Evaluation Criteria:**

Your assignment will be evaluated based on the following:

- Correctness of the Python code.
- Use of Avro for data serialization.
- Successful incremental fetching of data from the MySQL table.
- Successful writing of data to the Kafka topic with proper partitioning.
- Successful setup of a Kafka consumer group.
- Correct transformation and writing of data to separate JSON files.
- Quality of your documentation.

**Bonus Points:**

For bonus points, you can also provide:

- A Dockerfile to run your application.
- Unit tests for your Python code.
- Any advanced data processing, like filtering or transforming the data before sending it to the downstream system.

**Additional Instructions:**

Remember to update your database with new product details while testing the producer, so that it can fetch incremental data based on the last_updated timestamp.